🤖 AI CEO Automation Toolkit

Complete Technical Setup Guide for Autonomous Business Operations

$2,997 Value

Deploy AI agents for 24/7 business automation

📋 Table of Contents

1. Prerequisites & Requirements
2. OpenClaw Installation
3. AI Agent Deployment
4. Automation Workflows
5. Integration Setup
6. Monitoring & Maintenance
7. Security Configuration
8. Troubleshooting Guide
9. Advanced Configuration

10. Ongoing Maintenance

1. Prerequisites & System Requirements

1.1 Hardware Requirements

Component	Minimum	Recommended	Enterprise
CPU	2 cores	4 cores	8+ cores
RAM	4 GB	8 GB	16+ GB
Storage	20 GB SSD	50 GB SSD	100+ GB SSD
Network	1 Mbps	10 Mbps	100+ Mbps

1.2 Software Requirements

1 Operating System

Supported operating systems:

- [ ] Ubuntu 20.04+ (Recommended)
- [ ] macOS 11+ (Intel or Apple Silicon)
- [ ] Windows 10/11 with WSL2
- [ ] Debian 11+
- [ ] CentOS 8+

2 Required Software

```
# Node.js (v18 or higher) node --version # Should show v18.x or higher # npm (v9 or higher) npm --version # Should show v9.x or higher # Git (for repository management) git --version # Should show v2.x or higher # curl (for API testing) curl --version # Should be installed
```

3 Account Requirements

Before starting, you'll need accounts for:

- ☐ **API Provider** (choose one): OpenAI, Anthropic, Google AI, or local Ollama
- ☐ **Payment Processor** (optional): Stripe, Gumroad, or PayPal
- ☐ **Hosting** (optional): DigitalOcean, AWS, or local server
- ☐ **Messaging** (optional): Discord, Telegram, Slack

⚠️ **Important:** Never commit API keys or secrets to version control. Use environment variables or secure vault systems.

1.3 Time Investment

Setup Phase	Time Required	Skill Level
Prerequisites Check	15 minutes	Beginner
OpenClaw Installation	30-60 minutes	Beginner

Setup Phase	Time Required	Skill Level
Basic Agent Setup	1-2 hours	Intermediate
Advanced Workflows	2-4 hours	Intermediate
Full Deployment	4-6 hours	Advanced

2. OpenClaw Installation

2.1 Quick Installation (Recommended)

1 Download OpenClaw

```
# Install OpenClaw globally via npm npm install -g openclaw # Verify installation openclaw --version
```

2 Initialize Workspace

```
# Create workspace directory mkdir -p ~/.openclaw/workspace cd ~/.openclaw/workspace # Initialize configuration openclaw init
```

3 Configure API Keys

```
# Create secrets file (NEVER commit this) cat >
~/.openclaw/secrets.json << 'EOF' { "openai_api_key": "sk-your-
key-here", "anthropic_api_key": "sk-ant-your-key-here",
"google_ai_key": "your-key-here" } EOF # Set permissions
(read/write only by owner) chmod 600 ~/.openclaw/secrets.json
```

⚠️ **Security Warning:** The secrets.json file contains sensitive API keys. Never share this file or commit it to version control.

2.2 Manual Installation (Advanced)

1 Clone from Source

```
# Clone repository git clone
https://github.com/openclaw/openclaw.git cd openclaw # Install
dependencies npm install # Build from source npm run build #
Link globally npm link
```

2.3 Docker Installation

1 Run with Docker

```
# Pull latest image docker pull openclaw/openclaw:latest # Run container docker run -d \ --name openclaw \ -v ~/.openclaw:/root/.openclaw \ -p 18789:18789 \ openclaw/openclaw:latest
```

✅ **Installation Complete!** You should now be able to run `openclaw status` to verify your installation.

3. AI Agent Deployment

3.1 Understanding Agent Types

OpenClaw supports multiple agent types for different business functions:

Agent Type	Primary Function	Best For
Content Agent	Content creation & distribution	Marketing, blogging, social media
Revenue Agent	Income tracking & reporting	Financial monitoring, analytics
Support Agent	Customer service	Help desk, FAQ, ticket handling
Research Agent	Market analysis	Competitive intelligence, trends
Operations Agent	Workflow automation	Task scheduling, process management

3.2 Deploying Your First Agent

1 Create Agent Configuration

```
# Create agents directory mkdir -p ~/.openclaw/workspace/agents
# Create agent config cat >
~/.openclaw/workspace/agents/content-agent.yaml << 'EOF' name:
content-agent type: content model: openai/gpt-4 schedule: "0
6,12,18 * * *" # 6am, 12pm, 6pm daily tasks: - name:
"daily_content" description: "Generate daily content schedule"
template: "content/daily-schedule" - name: "social_posts"
description: "Create social media posts" template:
"content/social-posts" EOF
```

2 Activate Agent

```
# Deploy agent openclaw agent deploy content-agent.yaml #
Verify agent status openclaw agent status content-agent # View
agent logs openclaw agent logs content-agent --tail 50
```

3.3 Agent Configuration Options

📘 **Configuration Reference:**
Each agent configuration file supports the following options:

Parameter	Type	Description	Default
name	string	Unique agent identifier	Required

Parameter	Type	Description	Default
type	string	Agent category	Required
model	string	AI model to use	openai/gpt-4
schedule	cron	Execution schedule	None
tasks	array	List of tasks to execute	[]
memory	boolean	Enable persistent memory	true
tools	array	Available tools	all

3.4 Multi-Agent Deployment

1 Create Agency Configuration

```
# Create agency (multi-agent system) cat >
~/.openclaw/workspace/agency.yaml << 'EOF' name: ai-ceo-agency
agents: - name: lisa type: coordinator role: "CEO" model:
openai/gpt-4 - name: dwight type: research role: "Research
Lead" model: openai/gpt-4 - name: kelly type: content role:
"Content Creator" model: openai/gpt-3.5-turbo - name: pam type:
marketing role: "Marketing Lead" model: openai/gpt-4 workflows:
- name: content-pipeline steps: - agent: dwight task: research
- agent: kelly task: create-content - agent: pam task:
distribute EOF
```

2 Deploy Agency

```
# Deploy entire agency openclaw agency deploy agency.yaml # Monitor all agents openclaw agency status # Trigger workflow openclaw agency run content-pipeline
```

4. Automation Workflows

4.1 Workflow Design Principles

Before creating workflows, understand these core principles:

📘 **Key Principles:**

- **Start Simple:** Begin with 2-3 steps, add complexity later
- **Test Each Step:** Verify individually before chaining
- **Add Error Handling:** Always include failure notifications
- **Document Everything:** Future you will thank present you

4.2 Workflow Template Structure

```
# Workflow Template name: "workflow-name" description: "What this workflow does" trigger: type: "schedule|event|webhook" config: schedule: "0 9 * * *" # Daily at 9am conditions: all: - field: "status" equals: "approved" actions: - step: "step-1" action: "http-request" config: url: "https://api.example.com" method: "POST" - step: "step-2" action: "send-email" config: to: "user@example.com" template: "welcome" error_handling: - on_failure: action: "notify" config: channel: "slack" message: "Workflow failed: {error}"
```

4.3 Common Workflow Patterns

Pattern 1: Lead Capture to CRM

```
# Lead Capture Workflow name: "lead-to-crm" trigger: type: "webhook"
source: "form-submission" actions: - action: "validate-email" -
action: "add-to-crm" config: system: "airtable|hubspot|salesforce" -
action: "send-welcome-email" config: template: "welcome-sequence" -
action: "notify-team" config: channel: "slack"
```

Pattern 2: Content Distribution

```
# Content Distribution Workflow name: "content-distribution"
trigger: type: "schedule" config: schedule: "0 9 * * 1-5" # Weekdays
9am actions: - action: "fetch-content" config: source: "content-
calendar" - action: "generate-social-posts" config: platforms:
["twitter", "linkedin", "facebook"] - action: "schedule-posts"
config: tool: "buffer|hootsuite|later" - action: "log-distribution"
```

Pattern 3: Revenue Tracking

```
# Revenue Tracking Workflow name: "revenue-tracking" trigger: type:
"webhook" source: "stripe-payment" actions: - action: "parse-
payment" - action: "update-dashboard" config: system: "google-
sheets|airtable|notion" - action: "check-milestone" config:
threshold: [1000, 5000, 10000] - action: "notify-milestone" config:
channel: "discord|slack|telegram"
```

4.4 Workflow Testing Checklist

- [ ] Submit test data through trigger
- [ ] Verify each action executes correctly

- [ ] Check data maps to correct fields
- [ ] Test edge cases (empty fields, special characters)
- [ ] Verify error handling works
- [ ] Test with real data before production

5. Integration Setup

5.1 Payment Integration (Stripe)

1 Create Stripe Account

- Go to dashboard.stripe.com
- Complete account verification
- Get API keys from Developers → API Keys

2 Configure Stripe in OpenClaw

```
# Add Stripe credentials cat >
~/.openclaw/config/integrations/stripe.yaml << 'EOF' provider:
stripe api_key: ${STRIPE_SECRET_KEY} webhook_secret:
${STRIPE_WEBHOOK_SECRET} mode: live # or 'test' for sandbox
products: - id: prod_xxx name: "AI CEO Toolkit" price: 299700 #
$2,997 in cents currency: usd webhooks: - event:
"checkout.session.completed" action: "fulfill-order" - event:
"payment_intent.succeeded" action: "update-revenue" EOF
```

3 Test Stripe Integration

```
# Test with Stripe CLI stripe login stripe listen --forward-to localhost:18789/webhooks/stripe # Test webhook locally stripe trigger checkout.session.completed
```

⚠️ **Security:** Always use test mode during development. Never expose secret keys in code.

5.2 Messaging Integration (Discord)

1 Create Discord Bot

- Go to Discord Developer Portal
- Create New Application
- Navigate to Bot section
- Click "Add Bot"
- Copy the bot token

2 Configure Discord in OpenClaw

```
# Add Discord credentials cat >
~/.openclaw/config/integrations/discord.yaml << 'EOF' provider:
discord bot_token: ${DISCORD_BOT_TOKEN} client_id:
${DISCORD_CLIENT_ID} guild_id: ${DISCORD_GUILD_ID} channels:
notifications: "123456789" # Channel ID for notifications
revenue: "123456790" # Channel ID for revenue updates alerts:
"123456791" # Channel ID for system alerts permissions: -
send_messages - embed_links - read_message_history EOF
```

5.3 GitHub Integration

1 Create GitHub Token

- Go to GitHub Settings → Developer settings → Personal access tokens
- Generate new token (classic)
- Select scopes: repo, workflow, write:packages
- Copy and store securely

2 Configure GitHub in OpenClaw

```
# Add GitHub credentials cat >
~/.openclaw/config/integrations/github.yaml << 'EOF' provider:
github token: ${GITHUB_TOKEN} default_branch: main
repositories: - name: "ai-ceo-toolkit" owner: "your-username"
actions: - create_issue - create_pull_request - push_files
workflows: - name: "deploy-on-push" trigger: "push" branch:
"main" EOF
```

6. Monitoring & Maintenance

6.1 Health Checks

```
# Check OpenClaw status openclaw status # Check agent health
openclaw agent status --all # Check workflow status openclaw
workflow status --all # View recent logs openclaw logs --tail 100 #
Check resource usage openclaw stats
```

6.2 Automated Health Monitoring

1 Create Health Check Script

```
#!/bin/bash # health-check.sh - Run every hour via cron
openclaw status > /var/log/openclaw/health.log 2>&1 if [ $? -ne
0 ]; then echo "OpenClaw health check failed at $(date)" >>
/var/log/openclaw/errors.log openclaw restart fi
```

2 Add to Crontab

```
# Edit crontab crontab -e # Add health check (every hour) 0 * *
* * /home/user/.openclaw/scripts/health-check.sh
```

6.3 Log Management

```
# View logs in real-time openclaw logs --follow # Export logs for
analysis openclaw logs --export logs-$(date +%Y%m%d).json # Search
logs for errors openclaw logs --filter "error" --tail 100 # Archive
old logs (run weekly) find ~/.openclaw/logs -name "*.log" -mtime +7
-exec gzip {} \;
```

6.4 Backup Procedures

```
#!/bin/bash # backup.sh - Run daily
BACKUP_DIR="/backup/openclaw/$(date +%Y%m%d)" mkdir -p $BACKUP_DIR #
Backup configuration cp -r ~/.openclaw/config $BACKUP_DIR/ # Backup
workspace cp -r ~/.openclaw/workspace $BACKUP_DIR/ # Backup logs
(last 7 days) find ~/.openclaw/logs -name "*.log" -mtime -7 -exec cp
{} $BACKUP_DIR/ \; echo "Backup completed: $BACKUP_DIR"
```

7. Security Configuration

7.1 API Key Security

⚠️ **Critical Security Rules:**

- Never commit API keys to version control
- Use environment variables for secrets
- Rotate keys every 90 days
- Use separate keys for development/production

```
# Set environment variables (add to ~/.bashrc or ~/.zshrc) export
OPENAI_API_KEY="sk-your-key-here" export ANTHROPIC_API_KEY="sk-ant-
your-key-here" export STRIPE_SECRET_KEY="sk_live_your-key-here" #
Load environment variables source ~/.bashrc
```

7.2 Network Security

1 Configure Firewall

```
# Allow only localhost access (default) sudo ufw allow from 127.0.0.1 to any port 18789 # Or allow specific IP ranges sudo ufw allow from 192.168.1.0/24 to any port 18789 # Enable firewall sudo ufw enable
```

7.3 SSL/TLS Configuration

1 Generate SSL Certificate

```
# Install certbot sudo apt install certbot # Generate certificate sudo certbot certonly --standalone -d yourdomain.com # Certificate location # /etc/letsencrypt/live/yourdomain.com/fullchain.pem # /etc/letsencrypt/live/yourdomain.com/privkey.pem
```

2 Configure HTTPS in OpenClaw

```
# Update OpenClaw config cat > ~/.openclaw/config/server.yaml << 'EOF' server: host: 0.0.0.0 port: 18789 ssl: enabled: true cert: /etc/letsencrypt/live/yourdomain.com/fullchain.pem key: /etc/letsencrypt/live/yourdomain.com/privkey.pem EOF
```

8. Troubleshooting Guide

8.1 Common Issues & Solutions

Issue	Cause	Solution
Agent won't start	Missing API key	Check environment variables
Workflow fails silently	No error handling	Add error notifications
API rate limited	Too many requests	Add delays, use batching
Webhook not receiving	Wrong endpoint URL	Verify webhook configuration
Memory errors	Context too large	Clear memory, optimize prompts
Slow response	Network/model latency	Use faster model or cache

8.2 Diagnostic Commands

```
# Check system resources openclaw diagnostics --system # Check API connectivity openclaw diagnostics --api # Check agent status openclaw diagnostics --agents # Check workflow execution openclaw diagnostics --workflows # Full diagnostic report openclaw diagnostics --full
```

8.3 Error Code Reference

Code	Error	Description
E001	Authentication Failed	Invalid or missing API key
E002	Rate Limited	Exceeded API rate limits

Code	Error	Description
E003	Agent Not Found	Agent ID doesn't exist
E004	Workflow Failed	Workflow execution error
E005	Memory Overflow	Context window exceeded
E006	Network Error	Connection timeout or failure
E007	Configuration Error	Invalid config file
E008	Permission Denied	Insufficient permissions

9. Advanced Configuration

9.1 Multi-Node Deployment

1 Configure Node Network

```
# Primary node (master) cat >
~/.openclaw/config/nodes/master.yaml << 'EOF' role: master
bind: 0.0.0.0:18789 nodes: - id: node-1 host: 192.168.1.101
role: worker - id: node-2 host: 192.168.1.102 role: worker EOF
```

2 Configure Worker Nodes

```
# Worker node configuration cat >
~/.openclaw/config/nodes/worker.yaml << 'EOF' role: worker
master: 192.168.1.100:18789 capabilities: - content-generation
- web-research - data-processing EOF
```

9.2 Custom Agent Tools

1 Create Custom Tool

```
# Define custom tool cat > ~/.openclaw/tools/custom-tool.js <<
'EOF' module.exports = { name: 'custom-analytics', description:
'Custom analytics tool for business metrics', async
execute(params) { // Your custom logic here const metrics =
await fetchMetrics(params); return { success: true, data:
metrics, timestamp: new Date().toISOString() }; }, schema: {
params: { type: 'object', properties: { metric: { type:
'string' }, timeframe: { type: 'string' } }, required:
['metric'] } } }; EOF
```

2 Register Tool

```
# Register custom tool openclaw tool register custom-tool.js #
Verify registration openclaw tool list
```

9.3 Performance Tuning

Parameter	Default	Optimized	Notes
max_concurrent_agents	5	10-20	Based on CPU cores
memory_limit_mb	1024	2048-4096	Based on RAM
request_timeout_ms	30000	60000-120000	Based on model speed
cache_ttl_seconds	3600	7200-86400	Based on data freshness
batch_size	10	20-50	Based on API limits

10. Ongoing Maintenance

10.1 Daily Tasks

- ☐ Check system status (1 minute)
- ☐ Review error logs (2 minutes)
- ☐ Verify agent health (1 minute)
- ☐ Monitor revenue tracking (1 minute)

10.2 Weekly Tasks

- ☐ Review workflow performance (15 minutes)
- ☐ Check API usage and costs (5 minutes)
- ☐ Update outdated dependencies (10 minutes)

- [ ] Backup configuration and data (5 minutes)
- [ ] Test all integrations (10 minutes)

10.3 Monthly Tasks

- [ ] Rotate API keys (30 minutes)
- [ ] Security audit (1 hour)
- [ ] Performance optimization (1 hour)
- [ ] Update documentation (30 minutes)
- [ ] Review and clean up logs (15 minutes)
- [ ] Test disaster recovery (30 minutes)

10.4 Maintenance Schedule Template

```
# /etc/cron.d/openclaw-maintenance # Daily health check (6 AM) 0 6 * * * openclaw /home/user/.openclaw/scripts/health-check.sh # Weekly backup (Sunday 2 AM) 0 2 * * 0 openclaw /home/user/.openclaw/scripts/backup.sh # Monthly log rotation (1st of month, 3 AM) 0 3 1 * * openclaw /home/user/.openclaw/scripts/log-rotation.sh # Monthly security check (15th of month, 4 AM) 0 4 15 * * openclaw /home/user/.openclaw/scripts/security-check.sh
```

What You Get:

- ✅ Complete installation scripts
- ✅ 50+ automation templates
- ✅ Integration guides (Stripe, Discord, GitHub)
- ✅ Security hardening configurations
- ✅ Monitoring and maintenance tools
- ✅ Troubleshooting reference
- ✅ Advanced configuration options
- ✅ Lifetime updates

AI CEO Platform

$2,997

One-time payment • Complete system

Support: support@aiceosystems.com | **Community:** discord.gg/ai-ceo

© 2024 AI CEO Systems. All rights reserved.

Technical Support: support@aiceosystems.com
Documentation: docs.aiceosystems.com